

Lab Sheet 12: Legacy Code & Complex Refactoring

Objectives

By completing today's workshop, you will:

- Fetch a highly complex "legacy" module and wire it into your existing API.
- Manually identify **Code Smells** and **Vulnerabilities** (SQL Injection, High Cyclomatic Complexity).
- Achieve **100% Branch Coverage** using PyTest before altering any production code.
- Apply advanced refactoring techniques (Guard Clauses, Pydantic Models).
- Implement **Cloud Static Analysis** using GitHub Actions.

Introduction: Inheriting the Mess

In the real world, you rarely start from a blank slate. You usually inherit code written by developers who were rushing to meet deadlines. Today, we simulate this. A former team member wrote a "Mission Statistics" module. It works, but it suffers from high cyclomatic complexity, framework misuse, and duplicated logic. You will download this patch, wire it into your application, lock in its behavior with tests, and refactor it into clean, professional engineering.

Task 1: Fetching & Wiring the Legacy Code

Goal: Download the legacy patch and connect it to your main FastAPI application.

The Concept: Because you created your repository from a template, your Git history is detached from the module leader's updates. Instead of a messy Git merge, you will use the terminal to download the raw Python file directly from the source.

Action:

1. Open your Assessment 1 repository in VS Code (using the DevContainer).
2. Open your terminal, navigate into your backend folder, and download the legacy file using `curl`:

```
cd backend
curl -O \
  https://raw.githubusercontent.com/UoL-SoCS/CMP9134_2526_template/main/backend/legacy_stats.py
```

3. You now have `legacy_stats.py` in your folder. Do not edit it yet!

4. **Wire it up:** Open your `backend/main.py` file. You must attach this new module to your live server by adding two lines:
 - Near the top of the file, add: `from legacy_stats import router as legacy_router`
 - Further down (after `app = FastAPI()`), add: `app.include_router(legacy_router)`
5. Check your browser at `http://localhost:8000/docs`. You should see the new `/api/mission_stats` endpoint appear!
6. What does this endpoint do? Look at the `legacy_stats.py` file and read through the `calc_stats` function. You can use the "Try it out" feature in the Swagger UI to experiment with different inputs and see how it behaves. This will help you understand the code before you start refactoring it.

Task 2: Code Review (Identifying Smells)

Goal: Spot the architectural, logical, and framework flaws in the inherited snippet.

The Concept: Before changing code, you must understand why it is bad. Look for the Code Smells discussed in the lecture, as well as poor use of the FastAPI framework.

Action:

1. Create a new file called `DEBT_LOG.md` in the root of your repository.
2. Open `legacy_stats.py` and read through the `calc_stats` function.
3. Write down at least **four distinct "Code Smells"** you can spot. For each, explain *why* it is bad.
4. *Hints:*
 - Look at the `try/except KeyError` block. How should a FastAPI application handle incoming JSON validation automatically?
 - Look at the nested `if d > 0 and b > 0:` blocks. This is high "Cyclomatic Complexity."
 - Look at the database query string. What critical security flaw exists here?

Task 3: Locking Behavior (100% Branch Coverage)

Goal: Write regression tests to cover every possible path through the messy code.

Note: This assumes you have completed the previous workshop and have a working test suite set up with PyTest.

The Concept: Because this code is highly nested, a single "Happy Path" test is not enough. You must write tests that trigger the errors, the zero-division checks, and the score capping.

Action:

1. Open your `backend/tests/test_api.py` file.

2. Write at least **four different tests** for this specific endpoint. You must cover:
 - A successful Recon mission (Type 1).
 - A successful Transport mission with a heavy payload (Type 2, Payload > 50).
 - A request missing the "batt" key (Expect a 400 error).
 - A mission where the calculated score exceeds 100 (Verify it caps at 100).
3. Run `pytest tests/ -v` to ensure all tests pass. *Do not proceed until you have a green test suite!*

Task 4: Advanced Refactoring

Goal: Fix the code without breaking the application's behavior.

The Concept: Now that your tests are guarding you, you can tear the code apart.

Action:

1. **Fix the Validation:** Delete the `data: dict` parameter and the `try/except` block. Replace it with a Pydantic `BaseModel` class. Let FastAPI do the validation for you!
2. **Flatten the Nested Ifs:** Use "Guard Clauses." Check for invalid data upfront and return early, rather than nesting the success logic deep inside an `if` block.
3. **Extract the Duplication:** Move the score capping logic (`if score > 100: score = 100`) into a one-line helper function or use Python's `min()` function.
4. **Verify:** Run `pytest tests/` again. If the tests still pass, you successfully refactored a highly complex function into clean code!

Task 5: Cloud Static Analysis (Linting)

Goal: Force your GitHub Actions pipeline to reject messy code automatically.

The Concept: We can add a "Linter" (flake8) to your Continuous Integration pipeline. If a team member tries to push poorly formatted code with syntax errors or unused variables in the future, the pipeline will fail.

Action:

1. Open your existing GitHub Actions workflow file (e.g., `.github/workflows/ci.yml`).
2. Find the `test` job. Just before the step that runs `pytest`, add this new step:

```
- name: Static Analysis (Linting)
  run: |
    pip install flake8
    flake8 ./backend --count --max-complexity=5 --show-source --statistics
```

3. Commit your refactored code and this updated YAML file. Push to GitHub.
4. Check your Actions tab on GitHub. Your CI pipeline now acts as an automated guard against syntax-level Technical Debt!
5. Check for any linting errors in your Github Actions. If you see any, fix them and push again until the pipeline is green.

Further Resources

- **Refactoring.Guru - Guard Clauses:** refactoring.guru - *How to flatten nested if-statements.*
- **Pydantic Documentation:** docs.pydantic.dev - *How to replace manual dict parsing with robust models.*
- **Flake8 Command-Line Arguments:** flake8.pycqa.org/en/latest/user/options.html - *Official documentation for the flags used in Task 5 (like `--max-complexity` and `--select`).*